

01_fetch

June 29, 2026

1 01 — Fetch & smoke test

Polymarket Smart-Money Consensus engine · Step 1 of 3

This notebook verifies the public Polymarket data layer works, and demonstrates each call. All logic lives in `pmc.py` (the shared client + config) so the three notebooks stay thin.

Read-only. Nothing here trades. See Blueprint §11 for execution.

Run order: 01_fetch → 02_skill_roster → 03_consensus_signals.

```
[11]: # One-time setup (uncomment if needed)
      # %pip install requests pandas nbformat

      import importlib, pmc
      importlib.reload(pmc)
      from pmc import CFG, get_leaderboard, get_open_positions, get_closed_positions,
      ↪get_market, get_top_holders
      import pandas as pd
      print("Config OK. Bankroll:", CFG.BANKROLL, "| Kelly fraction:", CFG.
      ↪KELLY_FRACTION)
```

Config OK. Bankroll: 5000.0 | Kelly fraction: 0.25

1.1 1. Smoke test — does the leaderboard endpoint respond?

The leaderboard is the one endpoint whose host/path Polymarket has changed before. If this returns rows, you're good. If it returns an empty list, open `pmc.py` and either fix `CFG.LEADERBOARD_URL` or use the **top-holders fallback** in cell 4.

```
[12]: lb = get_leaderboard(window="all", limit=10)
      print(f"leaderboard rows: {len(lb)}")
      pd.DataFrame(lb).head(10)
```

leaderboard rows: 10

```
[12]:
```

	wallet	user \
0	0x56687bf447db6ffa42ffe2204a05edaa20f55839	Theo4
1	0x1f2dd6d473f3e824cd2f8a89d9c69fb96f6ad0cf	Fredi9999
2	0x204f72f35326db932158cba6adff0b9a1da95e14	swisstony
3	0x6a72f61820b26b1fe4d956e17b6dc2a1ea3033ee	kch123

4	0x2005d16a84ceefa912d4e380cd32e7ff827875ea	RN1
5	0x96cfc0c30942cfcdf76c7d408794d66b1acb	mintblade
6	0xed64a7bf029040aa331abc87902434d815ef217d	fishalive
7	0xbc11a64ab34a03a043fbe80598fa065ee87eeec6	frostrizz
8	0x78b9ac44a6d7d7a076c14e0ad518b301b63c6b76	Len9311238
9	0x664ce9fb97ae1bbd538d7381b2f4e92dab16f49c	sparklingwater123

	pnl	vol
0	2.205393e+07	4.301326e+07
1	1.661951e+07	7.661132e+07
2	1.463766e+07	1.303874e+09
3	1.138669e+07	2.937076e+08
4	1.023977e+07	7.787674e+08
5	9.238345e+06	1.775992e+07
6	9.063378e+06	1.328146e+07
7	8.928561e+06	2.309132e+07
8	8.709973e+06	1.640274e+07
9	8.474966e+06	1.900170e+07

1.2 2. Open positions for one wallet

Pick a wallet from the leaderboard above and inspect its live positions — these are the raw rows the consensus engine groups on.

```
[13]: wallet = lb[0]["wallet"] if lb else "0x0000000000000000000000000000000000000000000000000000000000000000"
pos = get_open_positions(wallet)
print(f"{wallet}: {len(pos)} open positions")
cols = ["title", "outcome", "size", "avgPrice", "curPrice", "currentValue",
        ↪ "cashPnl", "conditionId"]
df = pd.DataFrame(pos)
df[[col for col in cols if col in df.columns]].head(10) if len(df) else "no
        ↪ positions"
```

0x56687bf447db6ffa42ffe2204a05edaa20f55839: 0 open positions

```
[13]: 'no positions'
```

1.3 3. Closed positions → raw material for the skill score

We compute skill ourselves from resolved trades (Blueprint §3), not the leaderboard window.

```
[14]: closed = get_closed_positions(wallet)
print(f"{wallet}: {len(closed)} closed positions")
cdf = pd.DataFrame(closed)
show = [col for col in
        ↪ ["title", "outcome", "size", "avgPrice", "realizedPnl", "percentRealizedPnl"] if
        ↪ col in cdf.columns]
```

```
cdf[show].head(10) if len(cdf) else "no closed positions returned (check_↵
↵endpoint in pmc.py)"
```

0x56687bf447db6ffa42ffe2204a05edaa20f55839: 22 closed positions

```
[14]:
```

	title	outcome	avgPrice	\
0	Which party wins 2024 US Presidential Election?	Republican	0.579899	
1	Henry Cavill announced as next James Bond?	No	0.966000	
2	Henry Cavill announced as next James Bond?	Yes	0.118965	
3	Kamala Harris wins the popular vote?	No	0.374289	
4	Will Donald Trump win the popular vote in the ...	Yes	0.369503	
5	Will a Democrat win Pennsylvania Presidential ...	No	0.605411	
6	Will a Democrat win Michigan Presidential Elec...	No	0.544346	
7	Will a Democrat win Wisconsin Presidential Ele...	Yes	0.453806	
8	Will a Republican win Michigan Presidential El...	Yes	0.536793	
9	Will a Republican win Wisconsin Presidential E...	Yes	0.469997	


```

    realizedPnl
0  5.010862e+04
1 -1.042849e+02
2  8.530993e+01
3  6.061140e+06
4  8.303171e+06
5  8.434209e+04
6  1.725871e+04
7  1.052950e+03
8  5.612217e+03
9  2.915027e+04

```

1.4 4. (Backup) Build a candidate universe from top holders

If the leaderboard endpoint ever breaks, this always-public path seeds candidates: take a few high-volume markets and pull their largest holders. Optional.

```
[15]: # Example: paste a conditionId of a liquid market to see its top holders
# holders = get_top_holders("0x...")
# pd.DataFrame(holders).head(10)
print("Backup universe path ready - see get_top_holders() in pmc.py")
```

Backup universe path ready - see get_top_holders() in pmc.py

Next: 02_skill_roster.ipynb turns the candidate universe into a vetted roster.